

WHEN TO WRITE A KUBERNETES OPERATOR AND HOW TO DO IT RIGHT (FEATURING: TEA)

ABOUT ME

Michael Schmitt

Sitereliability Engineer at interhyp.de

Internet Hypothek (internet mortgage)

THE TRAP

**“WRITING A KUBERNETES OPERATOR
SOUNDS GREAT...”**

...until you have to maintain it.

SO WHEN *DOES* AN OPERATOR MAKE SENSE?

When you need:

- relationships between resources
- invariants (“if X exists, Y must exist”)
- lifecycle & cleanup
- continuous enforcement

WHEN YOU *DON'T* NEED ONE

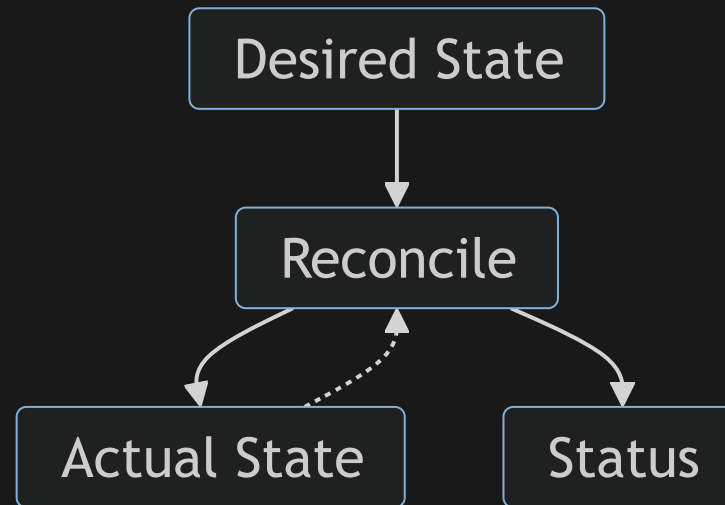
If it's:

- static resources
- one-off generation
- simple templating



Just use YAML.

NO MAGIC. JUST A LOOP.



WHAT MAKES A GOOD OPERATOR?

- simple reconciliation logic
- idempotent behavior
- clear, meaningful status
- boring, predictable outcomes

WHAT THE OPERATOR DOES NOT DO

- no workflows
- no step-by-step logic
- no hidden state

It only:

- observes
- corrects

THE LINE TO DRAW

You declare

- What you want with YAML

The Operator handles

- All relationships
- Complete Lifecycle
- Keeps everything in desired state
- Tells you when it fails

LET'S SCAFFOLD AN OPERATOR...

1. Install Go
2. Install a Kubernetes Distribution
3. Get the Operator SDK

LET'S SCAFFOLD AN OPERATOR...

```
operator-sdk init --domain "$DOMAIN" --repo "$REPO"
```

... AND CREATE A RESOURCE AND A CONTROLLER

```
operator-sdk create api \  
  --group kitchen \  
  --version v1alpha1 \  
  --kind Teapot \  
  --resource \  
  --controller
```

RECONCILE IS THE OPERATOR

```
package controller

/** ... */

func (r *TeapotReconciler) Reconcile(
    ctx context.Context,
    req ctrl.Request) (ctrl.Result, error) {
    // read current state
    // check dependencies
    // update status
    return ctrl.Result{}, nil
}
```

DEMO

TAKEAWAYS

- Operators are powerful - and expensive
- Write them rarely
- Keep them small
- Status is your UX

THANK YOU :-)



- [Presentation](#)
- [Golang](#)
- [Operator SDK](#)
- [Tea Pot Operator](#)
- [LinkedIn](#)